# 30-Day Python Coding Roadmap

## Easy to Advanced, BEL-Themed Problems (Radar, Sensors, Signals, Tracking)

One problem per day. Each problem has a description, sample input, sample output and a task. Write your own solution first, then check it against the sample output. Read all input using **input()** and print using **print()**.

| Day | Problem | Level |
|---|---|---|
| 1 | Radar Range from Echo Time | Easy |
| 2 | Sensor Reading Validator | Easy |
| 3 | Speed Unit Converter | Easy |
| 4 | Frequency Band Classifier | Easy |
| 5 | Log File Parser | Easy |
| 6 | Moving Average of Sensor Data | Easy |
| 7 | Mini Sensor Report (Combine Days 1-6) | Easy |
| 8 | Target Track List | Medium |
| 9 | Closest Target to Radar | Medium |
| 10 | Remove Duplicate Detections | Medium |
| 11 | Threat Priority Queue | Medium |
| 12 | Binary Message Packet Parser | Medium |
| 13 | Checksum Validator | Medium |
| 14 | Sliding Window Peak Detection | Medium |
| 15 | Distance and Bearing Between GPS Points | Medium-Hard |
| 16 | Polar to Cartesian for Radar Plot | Medium-Hard |
| 17 | Threshold Detection (Simple CA-CFAR) | Medium-Hard |
| 18 | Target Velocity from Two Positions | Medium-Hard |
| 19 | 1D Kalman Filter | Medium-Hard |
| 20 | Dominant Frequency using FFT | Medium-Hard |
| 21 | Low-Pass Noise Filter | Medium-Hard |
| 22 | Multi-Sensor Association (Nearest Neighbour) | Advanced |
| 23 | Finite State Machine for Radar Modes | Advanced |
| 24 | Task Scheduler with Deadlines | Advanced |
| 25 | Producer-Consumer Packet Stream | Advanced |
| 26 | Intrusion Zone Check (Point in Polygon) | Advanced |
| 27 | Grid Path Planning (A*) | Advanced |
| 28 | IoU and Non-Maximum Suppression | Advanced |
| 29 | Mini Tracking Pipeline | Advanced |
| 30 | Capstone: Surveillance System CLI | Advanced |

# Week 1: Basics (Easy)

## Day 1: Radar Range from Echo Time

**Description:**

A radar sends a pulse and receives the echo after time t. Range = (c x t) / 2, where c = 3 x $10^8$ m/s. Given echo times in microseconds, print the range of each target in metres and km.

| Input | Output |
|---|---|
| 3<br>100<br>250<br>40 | 15000.0 m 15.0 km<br>37500.0 m 37.5 km<br>6000.0 m 6.0 km |

**Task:**

Write a function **compute_range(t_us)** and a main loop that reads input and prints the results. First input line is the number of echoes, then one echo time (us) per line.

## Day 2: Sensor Reading Validator

**Description:**

A temperature sensor on a radar unit is valid only between -40 and 125 degrees C (both inclusive). For each reading print VALID or INVALID, then print the totals.

| Input | Output |
|---|---|
| 4<br>25.5<br>-60<br>130<br>0 | 25.5 VALID<br>-60.0 INVALID<br>130.0 INVALID<br>0.0 VALID<br>Valid: 2 Invalid: 2 |

**Task:**

Write **is_valid(value)** returning True/False. Read n, then n readings (floats).

## Day 3: Speed Unit Converter

**Description:**

Naval and air tracking uses knots. 1 knot = 1.852 km/h. Convert each speed to km/h and to m/s (divide km/h by 3.6). Print with 2 decimals.

| Input | Output |
|---|---|
| 3<br>10<br>25<br>100 | 10 kn = 18.52 km/h = 5.14 m/s<br>25 kn = 46.30 km/h = 12.86 m/s<br>100 kn = 185.20 km/h = 51.44 m/s |

**Task:**

Write **knots_to_kmh(k)** and **kmh_to_ms(v)**. Print the input value exactly as it was read.

## Day 4: Frequency Band Classifier

**Description:**

Classify each frequency (MHz) into a band. Lower limit inclusive, upper limit exclusive:
HF 3-30, VHF 30-300, UHF 300-1000, L 1000-2000, S 2000-4000, C 4000-8000, X 8000-12000. Anything else is UNKNOWN.

| Input | Output |
|---|---|
| 6<br>15<br>150<br>450<br>1300<br>9500<br>50000 | 15 MHz -> HF<br>150 MHz -> VHF<br>450 MHz -> UHF<br>1300 MHz -> L<br>9500 MHz -> X<br>50000 MHz -> UNKNOWN |

**Task:**

Write **classify_band(freq_mhz)** using a list of (low, high, name) tuples and a loop.

## Day 5: Log File Parser

**Description:**

A radar system writes log lines in the form **HH:MM:SS SEVERITY message**. Count how many lines exist for each severity: INFO, WARN, ERROR, CRITICAL.

| Input | Output |
|---|---|
| 6<br>10:00:01 INFO System started<br>10:00:05 WARN Low signal<br>10:01:10 ERROR Antenna fault<br>10:02:00 INFO Scan complete<br>10:03:30 WARN High temperature<br>10:04:00 CRITICAL Power loss | INFO: 2<br>WARN: 2<br>ERROR: 1<br>CRITICAL: 1 |

**Task:**

Use **str.split()** and a dictionary of counters. Always print all four severities in the order shown, even if the count is 0.

## Day 6: Moving Average of Sensor Data

**Description:**

Noisy sensor data is smoothed with a moving average of window size k. Output the average of every k consecutive values (n - k + 1 results), 2 decimals.

| Input | Output |
|---|---|
| 6 3<br>10 20 40 30 50 70 | 23.33 30.00 40.00 50.00 |

**Task:**

Write **moving_average(data, k)**. First line is n and k, second line has n values.

## Day 7: Mini Sensor Report (Combine Days 1-6)

**Description:**

Each line has: sensor id, echo time (us) and temperature (C). Print the range in km and temperature validity (-40 to 125 valid). At the end print the count of valid sensors and the average range (km) of the valid ones only.

| Input | Output |
|---|---|
| `4`<br>`R1 100 25`<br>`R2 250 140`<br>`R3 40 -10`<br>`R4 400 30` | `R1 15.0 km VALID`<br>`R2 37.5 km INVALID`<br>`R3 6.0 km VALID`<br>`R4 60.0 km VALID`<br>`Valid: 3`<br>`Average range (valid): 27.00 km` |

**Task:**

Reuse your functions from Day 1 and Day 2 and put them in a module **radar_utils.py**, then import it.

# Week 2: Data Structures (Medium)

## Day 8: Target Track List

**Description:**

A radar reports target updates as **ID x y**. Keep only the latest position of each target. After all updates print targets sorted by ID and the total count.

| Input | Output |
|---|---|
| `5`<br>`T1 10 20`<br>`T2 5 5`<br>`T1 12 22`<br>`T3 0 7`<br>`T2 6 8` | `T1 12 22`<br>`T2 6 8`<br>`T3 0 7`<br>`Total tracks: 3` |

**Task:**

Use a dictionary keyed by target ID. Print using **sorted(tracks)**.

## Day 9: Closest Target to Radar

**Description:**

The radar is at the origin (0, 0). Given targets with positions in km, find the one nearest to the radar and print its distance with 2 decimals.

| Input | Output |
|---|---|
| `4`<br>`A 3 4`<br>`B 1 1`<br>`C -6 8`<br>`D 2 -2` | `Closest: B at 1.41 km` |

**Task:**

Write **distance(x, y)** using math.sqrt and use **min()** with a key function.

## Day 10: Remove Duplicate Detections

**Description:**

The same target is often detected several times in one scan. Given grid-cell detections (x y), remove duplicates but keep the order of first appearance.

| Input | Output |
|---|---|
| 6<br>3 4<br>5 6<br>3 4<br>7 8<br>5 6<br>9 1 | Unique detections: 4<br>3 4<br>5 6<br>7 8<br>9 1 |

**Task:**

Use a **set** of tuples to remember what has been seen, and a list to keep the order.

## Day 11: Threat Priority Queue

**Description:**

Each target has a threat level (higher = more dangerous). Process targets in order of decreasing threat. For equal threat, the one that arrived first goes first.

| Input | Output |
|---|---|
| 5<br>T1 3<br>T2 9<br>T3 5<br>T4 9<br>T5 1 | T2 9<br>T4 9<br>T3 5<br>T1 3<br>T5 1 |

**Task:**

Use **heapq** with tuples of (-threat, arrival_index, id). Pop until empty.

## Day 12: Binary Message Packet Parser

**Description:**

A radar sends 8-byte packets (big-endian): header (1 byte, must be 0xAA), message type (1 byte), target id (2 bytes), range in metres (2 bytes), bearing in degrees (2 bytes). The packet is given as a hex string. If the header is wrong print **Invalid packet**.

| Input | Output |
|---|---|
| AA0100073A98005A | Type: 1<br>Target ID: 7<br>Range: 15000 m<br>Bearing: 90 deg |

**Task:**

Use **bytes.fromhex()** and **struct.unpack('>BBHHH', data)**.

## Day 13: Checksum Validator

**Description:**

The last byte of each packet is the checksum = (sum of all previous bytes) mod 256. Check each hex packet and report VALID or INVALID. For invalid ones show the expected and received checksum.

| Input | Output |
|---|---|
| `3`<br>`AA010007B2`<br>`AA010007B3`<br>`10203060` | `Packet 1: VALID`<br>`Packet 2: INVALID (expected B2, got B3)`<br>`Packet 3: VALID` |

**Task:**

Write **verify(packet_hex)**. Print checksums as 2-digit uppercase hex: **f'{v:02X}'**.

## Day 14: Sliding Window Peak Detection

**Description:**

A peak is a value strictly greater than both neighbours and above a given threshold. First and last samples are not checked (they have only one neighbour). Print every peak with its index.

| Input | Output |
|---|---|
| `10 50`<br>`10 30 60 40 20 55 80 45 30 70` | `Peak at index 2: 60`<br>`Peak at index 6: 80`<br>`Total peaks: 2` |

**Task:**

First line is n and threshold. Slide a window of 3 values over the data. Note that 55 is not a peak because 80 is larger.

# Week 3: Algorithms and Numerics (Medium-Hard)

## Day 15: Distance and Bearing Between GPS Points

**Description:**

Given two points (lat, lon in degrees), compute the great-circle distance using the Haversine formula (Earth radius 6371 km) and the initial bearing from point 1 to point 2 (0-360 degrees, clockwise from north).

| Input | Output |
|---|---|
| `12.9716 77.5946`<br>`13.0827 80.2707` | `Distance: 290.17 km`<br>`Bearing: 87.26 deg` |

**Task:**

Use math.radians, sin, cos, asin, atan2. Bearing = atan2(sin(dlon)*cos(lat2), cos(lat1)*sin(lat2) - sin(lat1)*cos(lat2)*cos(dlon)), then mod 360.

## Day 16: Polar to Cartesian for Radar Plot

**Description:**

Radar gives range r (km) and azimuth (degrees clockwise from north). Convert to x (east) and y (north): x = r sin(az), y = r cos(az). Print 2 decimals.

| Input | Output |
|---|---|
| `4`<br>`10 0`<br>`10 90`<br>`20 180`<br>`5 45` | `0.00 10.00`<br>`10.00 0.00`<br>`0.00 -20.00`<br>`3.54 3.54` |

**Task:**

Write **polar_to_xy(r, az_deg)**. Convert degrees to radians first.

## Day 17: Threshold Detection (Simple CA-CFAR)

**Description:**

Cell-Averaging CFAR: for each cell under test, take T training cells on each side, skipping G guard cells next to it. Threshold = alpha x mean(training cells). Detect if value > threshold. Only test cells that have full training windows on both sides.

| Input | Output |
|---|---|
| `12 2 1 3`<br>`2 3 2 3 2 40 3 2 3 2 3 2` | `Detection at index 5: value 40 (threshold 7.50)` |

**Task:**

First line is n, T, G, alpha. Test indices from G+T to n-G-T-1. Print one line per detection.

## Day 18: Target Velocity from Two Positions

**Description:**

Two radar measurements are given as **time(s) x(m) y(m)**. Compute speed (m/s and km/h) and heading in degrees clockwise from north: heading = atan2(dx, dy).

| Input | Output |
|---|---|
| `0 0 0`<br>`10 3000 4000` | `Speed: 500.00 m/s (1800.00 km/h)`<br>`Heading: 36.87 deg` |

**Task:**

Use math.hypot and math.atan2. Make sure heading is in the range 0-360.

## Day 19: 1D Kalman Filter

**Description:**

Estimate a constant value from noisy measurements. For each measurement z: predict P = P + Q; gain K = P / (P + R); update x = x + K (z - x); P = (1 - K) P. Print each estimate to 3 decimals.

| Input | Output |
| --- | --- |
| `0.01 1.0 0.0 1.0`<br>`5`<br>`1.0 1.2 0.9 1.1 1.05` | `0.502 0.739 0.781 0.848 0.885` |

**Task:**

First line: Q, R, initial x, initial P. Second line: n. Third line: n measurements. Write a **KalmanFilter** class with an **update(z)** method.

## Day 20: Dominant Frequency using FFT

**Description:**

Build a signal sampled at fs Hz for the given duration as the sum of two sine waves (frequency, amplitude). Use an FFT to find the two strongest frequencies.

| Input | Output |
| --- | --- |
| `100 1`<br>`5 1.0`<br>`20 0.5` | `Dominant frequency: 5.0 Hz`<br>`Second: 20.0 Hz` |

**Task:**

First line: fs and duration (s). Next lines: frequency and amplitude. Use **numpy.fft.rfft**, **rfftfreq** and **argsort** on the magnitude.

## Day 21: Low-Pass Noise Filter

**Description:**

Implement an exponential low-pass filter: y[0] = x[0], y[i] = alpha x[i] + (1 - alpha) y[i-1]. Print the filtered values with 2 decimals.

| Input | Output |
| --- | --- |
| `5 0.4`<br>`10 20 10 30 20` | `10.00 14.00 12.40 19.44 19.66` |

**Task:**

First line is n and alpha. Write **low_pass(data, alpha)** returning a list.

# Week 4: Advanced, System Level

## Day 22: Multi-Sensor Association (Nearest Neighbour)

**Description:**

Match each radar detection with the nearest unused camera detection within a gate distance. Process radar detections in input order. Report unmatched camera detections at the end.

| Input | Output |
|---|---|
| 5<br>3<br>R1 10 10<br>R2 50 50<br>R3 100 100<br>3<br>C1 12 11<br>C2 52 49<br>C3 200 200 | R1 -> C1 (2.24)<br>R2 -> C2 (2.24)<br>R3 -> no match<br>Unmatched camera: C3 |

**Task:**

First line is the gate distance. Then number of radar detections and their data, then number of camera detections and their data.

## Day 23: Finite State Machine for Radar Modes

**Description:**

States: OFF, STANDBY, SEARCH, TRACK, FAULT. Commands: POWER_ON (OFF to STANDBY), START (STANDBY to SEARCH), DETECT (SEARCH to TRACK), LOST (TRACK to SEARCH), STOP (SEARCH or TRACK to STANDBY), POWER_OFF (STANDBY to OFF), FAULT (any state to FAULT), RESET (FAULT to STANDBY). Any other command is invalid and the state does not change.

| Input | Output |
|---|---|
| 7<br>POWER_ON<br>START<br>DETECT<br>LOST<br>STOP<br>DETECT<br>POWER_OFF | POWER_ON: OFF -> STANDBY<br>START: STANDBY -> SEARCH<br>DETECT: SEARCH -> TRACK<br>LOST: TRACK -> SEARCH<br>STOP: SEARCH -> STANDBY<br>DETECT: INVALID in STANDBY<br>POWER_OFF: STANDBY -> OFF<br>Final state: OFF |

**Task:**

Store transitions in a dictionary **{(state, command): new_state}**. Handle FAULT separately.

## Day 24: Task Scheduler with Deadlines

**Description:**

A single processor runs tasks one after another starting at time 0, using Earliest Deadline First. Each task is **name duration deadline**. Print start time, finish time and whether the deadline was MET or MISSED.

| Input | Output |
|---|---|
| 4<br>T1 3 8<br>T2 2 4<br>T3 4 9<br>T4 1 5 | T2 start 0 finish 2 MET<br>T4 start 2 finish 3 MET<br>T1 start 3 finish 6 MET<br>T3 start 6 finish 10 MISSED<br>Deadlines missed: 1 |

**Task:**

Sort by deadline using **sorted(..., key=...)** and keep a running clock.

## Day 25: Producer-Consumer Packet Stream

**Description:**

A producer thread generates N packets with payloads 10, 20, ..., 10N and puts them in a queue. A consumer thread reads packets until it receives a stop signal (None) and adds up the payloads. The order of thread messages may vary, so only the final summary is checked.

| Input | Output |
| --- | --- |
| `5` | `Processed: 5 packets`<br>`Total payload: 150` |

**Task:**

Use **threading.Thread** and **queue.Queue**. The producer must put None at the end. Join both threads before printing the summary.

## Day 26: Intrusion Zone Check (Point in Polygon)

**Description:**

A restricted zone is a polygon. For each query point print INSIDE or OUTSIDE. Use the ray-casting method. Points never lie exactly on an edge.

| Input | Output |
| --- | --- |
| `6`<br>`0 0`<br>`10 0`<br>`10 5`<br>`5 5`<br>`5 10`<br>`0 10`<br>`3`<br>`2 8`<br>`8 8`<br>`8 2` | `INSIDE`<br>`OUTSIDE`<br>`INSIDE` |

**Task:**

Input: number of vertices, the vertices (L-shaped polygon here), number of queries, then the query points. Count how many polygon edges a horizontal ray crosses.

## Day 27: Grid Path Planning (A*)

**Description:**

A vehicle moves on a grid (4 directions) from S to G. # is an obstacle. Find the length of the shortest path. If there is no path print -1. Use A* with Manhattan distance as the heuristic.

| Input | Output |
| --- | --- |
| `5 5`<br>`S..#.`<br>`.#.#.`<br>`.#...`<br>`...#.`<br>`#...G` | `Shortest path length: 8` |

**Task:**

Use **heapq** for the open list and a dictionary for the best known cost of each cell.

## Day 28: IoU and Non-Maximum Suppression

**Description:**

Detections are boxes **x1 y1 x2 y2 score**. IoU = intersection area / union area. NMS: sort by score, keep the best box, remove all remaining boxes with IoU above the threshold with it, repeat.

| Input | Output |
| --- | --- |
| `4 0.5`<br>`10 10 50 50 0.9`<br>`12 12 52 52 0.8`<br>`100 100 150 150 0.7`<br>`11 9 49 51 0.6` | `Kept: 2`<br>`10 10 50 50 0.9`<br>`100 100 150 150 0.7` |

**Task:**

First line is the number of boxes and the IoU threshold. Write **iou(a, b)** and **nms(boxes, thr)**.

## Day 29: Mini Tracking Pipeline

**Description:**

Combine Days 8, 9 and 22. For every frame, match each detection to the nearest existing track within the gate (each track used once). Unmatched detections start new tracks with the next ID. Print the tracks seen in each frame.

| Input | Output |
| --- | --- |
| `5`<br>`3`<br>`2`<br>`10 10`<br>`50 50`<br>`2`<br>`12 11`<br>`52 52`<br>`3`<br>`14 12`<br>`54 54`<br>`100 100` | `Frame 1: ID1(10,10) ID2(50,50)`<br>`Frame 2: ID1(12,11) ID2(52,52)`<br>`Frame 3: ID1(14,12) ID2(54,54) ID3(100,100)`<br>`Total tracks: 3` |

**Task:**

First line is the gate, second the number of frames. For each frame: the number of detections, then one per line. Bonus: delete a track that is missed for 2 frames in a row.

## Day 30: Capstone: Surveillance System CLI

### Description:

Build a command-line surveillance simulator. The radar is at (0,0). Distances are in km. A target is in ALERT state if it is within 5 km, otherwise OK. Commands: ADD id x y, MOVE id x y, REMOVE id, LIST, NEAREST, REPORT. LIST uses insertion order. REPORT prints the number of targets and the number of alerts.

| Input | Output |
| --- | --- |
| `ADD T1 3 3`<br>`ADD T2 10 10`<br>`MOVE T2 2 2`<br>`LIST`<br>`NEAREST`<br>`REMOVE T1`<br>`REPORT` | `T1 added at (3, 3) dist 4.24 km ALERT`<br>`T2 added at (10, 10) dist 14.14 km OK`<br>`T2 moved to (2, 2) dist 2.83 km ALERT`<br>`T1 (3, 3) 4.24 km ALERT`<br>`T2 (2, 2) 2.83 km ALERT`<br>`NEAREST: T2 2.83 km`<br>`T1 removed`<br>`Targets: 1 Alerts: 1` |

### Task:

Use a **Target** class and a **SurveillanceSystem** class with one method per command. Read commands until end of input. Reuse code from earlier days and add basic error messages for unknown IDs.